

Jobsheet 3 — Responsive Design

Sub-CPMK: Membangun tampilan responsif.

Perubahan dari Jobsheet 2

- Tambah `<meta name="viewport">` di semua halaman.
- Navbar: hamburger menu memakai teknik **checkbox hack** murni CSS (`input[type=checkbox] + label`), aktif di layar $\leq 480\text{px}$.
- Tabel dibungkus `<div class="table-responsive">` agar bisa di-scroll horizontal di layar sempit.
- Tambah media query di `style.css`: grid kartu statistik 3 $\rightarrow$ 2 $\rightarrow$ 1 kolom mengikuti breakpoint tablet/mobile.

Cara menjalankan

Buka `index.html` di browser, uji dengan DevTools responsive mode pada 3 breakpoint (mobile $\leq 480\text{px}$, tablet $\sim 768\text{px}$, desktop $\geq 1024\text{px}$).

Catatan

- Hamburger di jobsheet ini masih murni CSS (checkbox hack). Di Jobsheet 5 akan diganti dengan toggle berbasis JavaScript.

Dokumentasi Jobsheet 3 — Responsive Design

Dokumentasi ini melanjutkan [dokumentasi jobsheet-02](#) dan ditujukan untuk mahasiswa yang baru belajar HTML/CSS. Kalau kamu belum paham HTML dasar (jobsheet-01) atau CSS dasar — Flexbox, Grid, box model (jobsheet-02) — sebaiknya baca dulu dokumentasi jobsheet sebelumnya karena bab-bab di sini akan sering merujuk balik ke sana.

Apa yang Baru di Jobsheet 3?

Sesuai [README.md](#) jobsheet ini, ada 4 penambahan dari jobsheet-02:

1. Tag `<meta name="viewport">` di semua halaman.

2. Menu **hamburger** (ikon ☰) di layar sempit, dibuat murni dengan CSS memakai teknik **checkbox hack** (tanpa JavaScript sama sekali).
3. Tabel dibungkus `<div class="table-responsive">` supaya bisa di-scroll ke samping di layar sempit, alih-alih memampatkan kolom sampai tidak terbaca.
4. **Media query** di `style.css` yang mengubah grid kartu statistik dari 3 kolom → 2 kolom → 1 kolom mengikuti lebar layar.

Semua perubahan ini masuk dalam topik besar **Responsive Web Design** — membuat satu halaman web yang tampilannya menyesuaikan diri secara otomatis di berbagai ukuran layar (HP, tablet, laptop, monitor besar), tanpa perlu membuat halaman terpisah untuk tiap perangkat.

Daftar Isi

1. [Konsep Dasar Responsive Design](#)
2. [Apa yang Berubah di File HTML?](#)
3. [CSS: Menu Hamburger dengan Checkbox Hack](#)
4. [CSS: Tabel yang Bisa Di-scroll \(`table-responsive` \)](#)
5. [CSS: Media Query & Breakpoint](#)
6. [Rangkuman & Latihan Lanjutan](#)

Struktur Folder

```
jobsheet-03/
├── index.html
├── assets/
│   └── css/
│       └── style.css      # Ditambah checkbox hack + media query
├── buku/
│   ├── list.html         # Tabel dibungkus .table-responsive
│   └── tambah.html
├── anggota/
│   ├── list.html         # Tabel dibungkus .table-responsive
│   └── tambah.html
├── README.md
└── Dokumentasi/          # Folder dokumentasi ini
```

Silakan baca urut dari nomor 1, atau langsung loncat ke bagian yang ingin dipahami.

1. Konsep Dasar Responsive Design

Sebelum membedah kode, kenali dulu istilah-istilah besar di balik "tampilan responsif" yang akan terus muncul di bab-bab berikutnya.

1.1 Apa itu Responsive Web Design?

Responsive Web Design (RWD) adalah pendekatan membangun halaman web agar **tata letaknya menyesuaikan diri secara otomatis** dengan lebar layar perangkat yang dipakai — HP, tablet, laptop, atau monitor besar — memakai **satu** file HTML/CSS yang sama, tanpa perlu membuat versi terpisah untuk tiap perangkat (misalnya `m.situs.com` khusus HP yang dulu populer sebelum era RWD).

Sebelum jobsheet ini, halaman `index.html` dkk. sebenarnya **sudah** "jalan" di HP (browser tetap menampilkannya), tapi belum **responsif** — karena tanpa `<meta name="viewport">` (dibahas di §1.2), browser HP akan menganggap halaman itu dirancang untuk layar desktop, lalu memperkecil (zoom out) seluruh halaman supaya "muat" di layar kecil — hasilnya teks jadi sangat kecil dan pengguna harus pinch-zoom manual.

1.2 `<meta name="viewport">` — Supaya Browser HP Tidak "Berbohong"

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Ini ditambahkan di `<head>` setiap halaman jobsheet ini (lihat detail perubahannya di [bab 2](#)). Tanpa baris ini, browser mobile **berpura-pura** memiliki lebar layar sekitar 980px (disebut *default layout viewport*) lalu men-zoom-out hasil render-nya supaya muat di layar sungguhan yang jauh lebih sempit — inilah sebabnya banyak situs lama terlihat "mini" dan butuh di-zoom manual saat dibuka di HP.

Atribut `content` berisi dua instruksi:

Instruksi	Artinya
<code>width=device-width</code>	Lebar viewport (area tampilan) disamakan dengan lebar layar fisik perangkat, bukan lebar default 980px tadi.
<code>initial-scale=1</code>	Level zoom awal diatur ke 1:1 (tidak di-zoom out ataupun in) saat halaman pertama kali dimuat.

Dengan dua instruksi ini, 1 piksel CSS (`px` yang kamu tulis di `style.css`) kurang lebih sama dengan 1 piksel yang benar-benar terlihat proporsional di layar HP, sehingga **media query** (dibahas selanjutnya) bisa bekerja secara akurat berdasarkan lebar layar yang sesungguhnya.

1.3 Media Query: CSS yang "Bertanya" Dulu

Media query adalah fitur CSS yang memungkinkan sekumpulan aturan CSS hanya berlaku **jika kondisi tertentu terpenuhi** — paling umum, kondisinya adalah lebar layar (viewport).

Sintaksnya:

```
@media (max-width: 768px) {  
  /* aturan CSS di sini HANYA berlaku  
    kalau lebar layar 768px atau lebih sempit */  
}
```

- `@media` — kata kunci yang memulai blok media query.
- `(max-width: 768px)` — **kondisi**: berlaku selama lebar viewport **kurang dari atau sama dengan** 768px.
- Semua aturan CSS di dalam kurung kurawal `{ }` hanya "aktif" ketika kondisi itu benar. Begitu layar diperlebar melebihi 768px (misalnya jendela browser di-resize), aturan di dalamnya otomatis **berhenti** berlaku — tidak butuh JavaScript maupun reload halaman sama sekali, murni kemampuan CSS.

Penjelasan detail tiap breakpoint yang dipakai jobsheet ini ada di [bab 5](#).

1.4 Breakpoint: Titik Ambang Batas

Breakpoint adalah nilai lebar layar tertentu yang dipakai sebagai "garis batas" di media query — di jobsheet ini ada 2 breakpoint:

Breakpoint	Nilai	Kira-kira Mewakili
Tablet	768px	Tablet dan layar sempit lainnya
Mobile	480px	HP

Angka-angka ini **bukan aturan baku universal** — banyak proyek memilih breakpoint berbeda tergantung kebutuhan desainnya. 768px dan 480px hanya kebiasaan umum yang mendekati lebar rata-rata perangkat tablet dan HP.

1.5 Pendekatan "Desktop-First" yang Dipakai di Jobsheet Ini

Ada dua strategi umum menulis CSS responsif:

Strategi	Cara Kerja
Desktop-first	Tulis dulu gaya default untuk layar besar, lalu timpa sebagian gaya itu di dalam <code>@media (max-width: ...)</code> untuk layar yang lebih sempit.
Mobile-first	Kebalikannya: tulis dulu gaya default untuk layar sempit , lalu tambahkan lebih banyak gaya di dalam <code>@media (min-width: ...)</code> untuk layar yang lebih lebar.

Jobsheet ini memakai pendekatan **desktop-first**: perhatikan seluruh gaya dari [jobsheet-02](#) (grid 3 kolom, navbar horizontal, dst.) tetap menjadi **gaya default/dasar**, dan `style.css` di jobsheet ini hanya **menambahkan** dua blok `@media (max-width: ...)` di bagian **paling bawah** file untuk menyimpannya di layar sempit (lihat [bab 5](#)).

Kenapa harus di bagian bawah file? Karena kalau dua aturan CSS punya spesifisitas selector yang sama (ingat konsep spesifisitas dari [dokumentasi jobsheet-02](#)), aturan yang ditulis **belakangan** di file yang menang. Kalau blok `@media` diletakkan **sebelum** gaya default, gaya default yang ditulis sesudahnya justru akan menimpa balik hasil media query — sehingga efek responsifnya "hilang" secara tidak sengaja. Ini kesalahan umum pemula saat pertama kali menambahkan media query ke stylesheet yang sudah ada.

Dengan bekal istilah-istilah di atas, sekarang kamu siap membaca perubahan konkretnya mulai bab 2.

Lanjut ke: [Apa yang Berubah di File HTML?](#)

2. Apa yang Berubah di File HTML?

Sama seperti transisi jobsheet-01 → jobsheet-02 ([lihat penjelasan pola ini](#)), struktur besar HTML jobsheet-03 **tidak berubah** dari jobsheet-02. Ada 3 penambahan kecil namun penting di HTML, ditambah beberapa baris CSS baru murni di `style.css` (dibahas mulai [bab 3](#)).

2.1 `<meta viewport>`

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Ditambahkan di `<head>` **semua 5 halaman** HTML, tepat di bawah `<meta charset="UTF-8">`. Penjelasan lengkap kenapa baris ini penting ada di [konsep dasar §1.2](#).

2.2 Pasangan Checkbox + Label untuk Hamburger Menu

```

<header>
  <h1>SIMPUS-Mini</h1>
  <input type="checkbox" id="nav-toggle" class="nav-toggle">
  <label for="nav-toggle" class="nav-toggle-label">&#9776;</label>
  <nav>
    ...
  </nav>
</header>

```

Dua elemen baru muncul di dalam `<header>`, **sebelum** `<nav>`:

- `<input type="checkbox" id="nav-toggle" class="nav-toggle">` — sebuah kotak centang (checkbox) biasa. Tapi lewat CSS (dibahas di [bab 3](#)), checkbox ini akan **disembunyikan** dari tampilan — ia dipakai bukan untuk dicentang secara visual, melainkan sekadar untuk **menyimpan status** "menu sedang terbuka atau tertutup" (dicentang = terbuka).
- `<label for="nav-toggle" class="nav-toggle-label">☰</label>` — ingat dari [dokumentasi jobsheet-01](#), atribut `for="nav-toggle"` **menghubungkan** label ini ke checkbox yang `id`-nya `nav-toggle`. Efeknya: **mengklik label ini sama saja** dengan mengklik checkbox itu sendiri** — inilah dasar dari teknik** "checkbox hack" yang dijelaskan lebih detail di [bab 3](#).
- `☰` — ini adalah **HTML entity** berbasis kode numerik (mirip `©` atau `—` yang sudah dibahas di [dokumentasi jobsheet-01](#)), menampilkan karakter simbol `≡` (tiga garis horizontal, ikon "hamburger" yang lazim dipakai untuk menu di layar sempit).

Penting: ketiga elemen ini (`h1`, `input`, `label`) semuanya berada **langsung di dalam** `<header>`, sejajar dengan `<nav>` — bukan di dalam `<nav>`. Susunan ini sengaja dipilih supaya CSS bisa memanfaatkan **sibling combinator** (`~`) antara checkbox dan `<nav>`, dijelaskan detail mekanismenya di [bab 3](#).

2.3 Pembungkus `<div class="table-responsive">`

Di `buku/list.html` dan `anggota/list.html`, tabel yang sebelumnya berdiri sendiri (lihat [dokumentasi jobsheet-01](#)) sekarang dibungkus satu `<div>` tambahan:

```

<div class="table-responsive">
  <table>
    ...
  </table>
</div>

```

`<div>` di sini dipakai murni sebagai **pembungkus teknis** untuk keperluan styling (`overflow-x: auto`, dibahas di [bab 4](#)) — beda dengan tag semantic (`header`, `section`, `article`, dst. dari [dokumentasi jobsheet-01](#)) yang membawa makna konten. `<div>` memang **sengaja tidak** punya makna semantic — ia hanya "kotak polos" yang dipakai ketika kita butuh target CSS/JS

tanpa perlu tag khusus. Class `table-responsive` di sini adalah nama yang **kita buat sendiri** (bukan nama bawaan HTML), dipilih supaya mudah dibaca maksudnya: "tabel yang sudah dibuat responsif".

Lanjut ke: [CSS: Menu Hamburger dengan Checkbox Hack](#)

3. CSS: Menu Hamburger dengan Checkbox Hack

Ini bagian paling "ajaib" di jobsheet ini: menu yang bisa dibuka-tutup dengan diklik, **tanpa satu baris JavaScript pun**. Triknya disebut **checkbox hack**, memanfaatkan pseudo-class `:checked` dan **sibling combinator**** CSS.**

3.1 Potongan CSS yang Terlibat

Kode ini tersebar di 2 tempat berbeda di `style.css`: gaya dasar (selalu aktif) dan gaya di dalam media query mobile (hanya aktif di layar sempit).

Gaya dasar (di luar media query):

```
/* ===== Hamburger Menu (checkbox hack) ===== */
.nav-toggle {
  display: none;
}

.nav-toggle-label {
  display: none;
  font-size: 1.6rem;
  color: #fff;
  cursor: pointer;
}
```

Di dalam `@media (max-width: 480px)` (dibahas lengkap di [bab 5](#)):

```
header {
  position: relative;
}

.nav-toggle-label {
  display: block;
}

header nav {
  display: none;
}
```

```

width: 100%;
order: 3;
margin-top: 1rem;
}

.nav-toggle:checked ~ nav {
display: block;
}

header nav ul {
flex-direction: column;
gap: 0.75rem;
}

```

3.2 Langkah 1 – Sembunyikan Checkbox Aslinya

```

.nav-toggle {
display: none;
}

```

Checkbox HTML aslinya (kotak kecil yang bisa dicentang) **disembunyikan** total** dengan `display: none`. Ingat dari HTML-nya** ([bab 2 §2.2](#)), checkbox ini memang tidak dimaksudkan untuk terlihat — ia hanya dipakai sebagai "penyimpan status" tersembunyi: **dicentang = menu terbuka**, tidak dicentang = menu tertutup**. Status ini yang nanti "dibaca" oleh** CSS lain lewat pseudo-class `:checked` di [§3.5](#).

3.3 Langkah 2 – Label Berperan sebagai Tombol Pengganti

```

.nav-toggle-label {
display: none; /* disembunyikan di layar besar */
font-size: 1.6rem;
color: #fff;
cursor: pointer;
}

```

Karena checkbox aslinya disembunyikan, `<label>` (berisi ikon ☰ dari [bab 2](#)) yang tampil menggantikannya sebagai "tombol" yang bisa diklik pengguna. Ingat hubungan `<label for="nav-toggle">` dengan `<input id="nav-toggle">` sudah dijelaskan konsepnya di [dokumentasi jobsheet-01](#): **mengklik label = mengklik checkbox-nya**, meskipun checkbox itu sendiri tidak terlihat sama sekali di layar. `cursor: pointer` menandakan label ini bisa diklik (sama seperti tombol biasa, lihat [dokumentasi jobsheet-02 §7.6](#)).

Perhatikan `.nav-toggle-label` diberi `display: none` di gaya dasar — artinya di layar **besar** (desktop/tablet), ikon hamburger ini **tidak** tampil sama sekali**, karena menu navigasi

memang sudah langsung terlihat** sejajar horizontal (lihat [dokumentasi jobsheet-02](#)). Ikon ini baru dimunculkan (`display: block`) khusus di dalam `@media (max-width: 480px)` — dijelaskan alurnya di [§3.6](#).

3.4 Langkah 3 — Sembunyikan Menu di Layar Sempit (Dulu)

```
header nav {  
  display: none;  
  width: 100%;  
  order: 3;  
  margin-top: 1rem;  
}
```

Aturan ini **hanya berlaku di dalam** `@media (max-width: 480px)` — di layar sempit, `<nav>` (menu navigasi lengkap) disembunyikan **secara default****, supaya tidak memenuhi layar HP yang kecil. `order: 3` adalah** properti Flexbox (ingat `<header>` adalah flex container dari [dokumentasi jobsheet-02 §4.3](#)) — mengatur `<nav>` supaya **selalu tampil paling akhir/bawah** di antara flex item lain (`h1`, checkbox tersembunyi, label) ketika ia memang ditampilkan nanti, alih-alih tersisip di tengah.

3.5 Langkah 4 — Sibling Combinator: Menghubungkan Status ke `nav`

```
.nav-toggle:checked ~ nav {  
  display: block;  
}
```

Ini **inti** dari seluruh trik checkbox hack. Mari bedah selector ini bagian per bagian:

Bagian	Artinya
<code>.nav-toggle</code>	Elemen dengan class <code>nav-toggle</code> — yaitu checkbox kita.
<code>:checked</code>	Pseudo-class (ingat dari dokumentasi jobsheet-02 §1.4): berlaku hanya saat elemen itu dalam kondisi tercentang.
<code>~</code>	General sibling combinator — selector baru yang belum pernah dibahas di jobsheet sebelumnya. Artinya "pilih elemen <code>nav</code> yang merupakan saudara (sibling) dari <code>.nav-toggle</code> , dan muncul setelahnya di HTML."

Bagian	Artinya
nav	Elemen target yang gayanya diubah.

Jadi selector lengkap `.nav-toggle:checked ~ nav` berarti: "**kalau** checkbox `.nav-toggle` sedang tercentang, ubah gaya elemen `<nav>` yang posisinya sejajar (sibling) setelahnya."** Inilah kenapa susunan HTML di** [bab 2 §2.2](#) penting: `input`, `label`, dan `nav` harus sama-sama berada **langsung** di dalam `<header>` (sibling satu sama lain), bukan `nav` di dalam elemen lain — sibling combinator **hanya** bekerja antar elemen yang levelnya sejajar.

Alur lengkapnya:

1. Pengguna klik label (ikon ☰).
2. Karena `<label for="nav-toggle">`, klik ini **mencentang** checkbox tersembunyi `#nav-toggle`.
3. Checkbox kini cocok dengan `:checked`.
4. Selector `.nav-toggle:checked ~ nav` menjadi aktif → `<nav>` (sebelumnya `display: none` dari [§3.4](#)) berubah jadi `display: block` → menu **muncul**.
5. Klik label sekali lagi → checkbox tidak lagi tercentang → aturan `:checked ~ nav` tidak berlaku lagi → `<nav>` kembali ke `display: none` dari aturan dasarnya → menu **tersembunyi lagi**.

Semua perpindahan status ini murni ditangani browser lewat CSS, tanpa event listener atau kode JavaScript apa pun — makanya disebut teknik "checkbox hack": **meminjam** perilaku bawaan checkbox (bisa dicentang/tidak, dan punya pseudo-class `:checked`) untuk keperluan yang sebenarnya di luar konteks form biasa.

3.6 Mengapa Aturan Ini Perlu Diulang di Dalam Media Query?

```
header {
  position: relative;
}

.nav-toggle-label {
  display: block;
}
```

Perhatikan `.nav-toggle-label` muncul **dua kali** di `style.css`: sekali di gaya dasar (`display: none;`, [§3.3](#)), sekali lagi di dalam `@media (max-width: 480px)` (`display: block;`). Ini **bukan** duplikasi yang salah — ini pola umum di CSS responsif desktop-first (ingat [konsep dasar §1.5](#)): gaya dasar menyembunyikan ikon hamburger di layar besar, lalu media query **menampilkannya kembali** menjadi terlihat khusus saat layar $\leq 480\text{px}$. Karena aturan di dalam `@media` ditulis **setelah** gaya dasar di file (dan spesifisitas keduanya sama, sama-sama satu class selector), aturan di dalam media query yang menang saat kondisinya terpenuhi.

`header { position: relative; }` juga hanya diaktifkan di breakpoint mobile ini — properti `position: relative` diperlukan supaya elemen `<nav>` (yang nanti akan diberi `width: 100%` saat terbuka) mengukur lebarnya relatif terhadap `<header>`, bukan elemen lain di luar yang mungkin ukurannya berbeda.

3.7 Sentuhan Terakhir: Menu Vertikal saat Terbuka

```
header nav ul {  
  flex-direction: column;  
  gap: 0.75rem;  
}
```

Ingat dari [dokumentasi jobsheet-02](#), `header nav ul` sudah menjadi flex container (`display: flex`) sejak `jobsheet-02`, menyusun item menu **horizontal** secara default. Properti `flex-direction: column` di dalam media query mobile ini mengubah arah susunan flex item jadi **vertikal** — masuk akal karena di layar HP yang sempit, menu yang terbuka lebih nyaman ditampilkan sebagai daftar bertumpuk ke bawah daripada dipaksa sejajar horizontal yang akan terlalu sempit per itemnya.

Lanjut ke: [CSS: Tabel yang Bisa Di-scroll \(table-responsive\)](#).

4. CSS: Tabel yang Bisa Di-scroll (table-responsive)

Bab pendek ini membahas solusi untuk masalah klasik: tabel dengan banyak kolom (seperti Daftar Buku dari [dokumentasi jobsheet-01](#)) akan terlihat sangat sempit dan sulit dibaca kalau dipaksa muat di layar HP.

4.1 Kode CSS

```
/* ===== Tabel Responsif ===== */  
.table-responsive {  
  overflow-x: auto;  
}
```

Hanya **satu properti**, tapi cukup untuk menyelesaikan masalahnya.

4.2 Dua Pilihan Menghadapi Tabel Lebar di Layar Sempit

Ada dua strategi umum menangani tabel lebar di layar sempit:

1. **Memampatkan isi tabel** — memaksa semua kolom mengecil supaya muat di layar sempit. Masalahnya: teks jadi terpotong/tidak terbaca, terutama untuk kolom seperti "Pengarang" yang berisi nama panjang.
2. **Membiarkan tabel selebar aslinya, tapi bisa di-scroll ke samping** — inilah pendekatan yang dipilih jobsheet ini. Lebar kolom tetap nyaman dibaca, dan pengguna tinggal geser (swipe di HP, atau scroll horizontal di trackpad/mouse) untuk melihat kolom yang belum terlihat.

4.3 Kenapa Perlu Dibungkus `<div>`?

Ingat dari [bab 2 §2.3](#), `<table>` dibungkus tambahan `<div class="table-responsive">`. Ini karena `overflow-x: auto` **tidak bisa diterapkan langsung** ke elemen `<table>` seefektif kalau diterapkan ke sebuah `<div>` pembungkus:

- `<table>` yang lebar (isinya banyak kolom) akan tetap **mendorong** melebar** kotak pembungkusnya sendiri, alih-alih terpotong dan** memicu scrollbar pada dirinya sendiri.
- Dengan membungkusnya dalam `<div class="table-responsive">` yang diberi `overflow-x: auto`, `<div>` inilah yang punya **lebar tetap** (mengikuti kotak `<section>` pembungkusnya — lihat [dokumentasi jobsheet-02 §5.3](#)), sementara `<table>` di dalamnya boleh melebar sesukanya. Kalau lebar `<table>` melebihi lebar `<div>` pembungkusnya, barulah `overflow-x: auto` menampilkan **scrollbar horizontal** otomatis, khusus di dalam area tabel itu saja — bukan men-scroll seluruh halaman.

4.4 Nilai `auto` pada `overflow-x`

```
overflow-x: auto;
```

- `overflow-x` mengatur perilaku konten yang melebihi lebar elemen, **khusus arah horizontal** (ada juga `overflow-y` untuk arah vertikal, dan `overflow` untuk keduanya sekaligus).
- Nilai `auto` berarti: browser **hanya menampilkan** scrollbar kalau memang dibutuhkan (isi di dalamnya melebihi lebar kotak). Kalau tabelnya cukup sempit untuk muat (misalnya di layar desktop lebar), tidak ada scrollbar yang muncul sama sekali — perilaku ini lebih ramah dibanding nilai `scroll` yang akan **selalu** menampilkan scrollbar meskipun tidak diperlukan.

4.5 Kapan Efek Ini Benar-Benar Terlihat?

Berbeda dengan menu hamburger ([bab 3](#)) yang baru aktif di breakpoint mobile, aturan `.table-responsive` ini **selalu aktif** di semua ukuran layar (tidak dibungkus `@media` sama sekali) — hanya saja efeknya **tidak terlihat** di layar lebar karena tabelnya memang sudah cukup sempit untuk

muat tanpa perlu scroll. Efek scroll horizontal baru "kelihatan" ketika lebar layar (dan karenanya lebar `<section>` pembungkus dari [dokumentasi jobsheet-02](#)) menyempit lebih kecil dari lebar total tabel (5 kolom Daftar Buku, atau 5 kolom Daftar Anggota).

Lanjut ke: [CSS: Media Query & Breakpoint](#)

5. CSS: Media Query & Breakpoint

Bab ini membahas dua blok `@media` di bagian **paling bawah** `style.css` — tempat sebagian besar "keajaiban" responsif jobsheet ini benar-benar terjadi.

5.1 Kode CSS Lengkap

```
/* ===== Responsive Breakpoints ===== */

/* Tablet ke bawah */
@media (max-width: 768px) {
  main section:nth-of-type(2) {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* Mobile */
@media (max-width: 480px) {
  header {
    position: relative;
  }

  .nav-toggle-label {
    display: block;
  }

  header nav {
    display: none;
    width: 100%;
    order: 3;
    margin-top: 1rem;
  }

  .nav-toggle:checked ~ nav {
    display: block;
  }

  header nav ul {
    flex-direction: column;
    gap: 0.75rem;
  }
}
```

```

}

main section:nth-of-type(2) {
  grid-template-columns: 1fr;
}

form input,
form select {
  max-width: 100%;
}
}

```

5.2 Breakpoint 1: Tablet (768px ke Bawah) — Grid 3 Kolom Jadi 2 Kolom

```

@media (max-width: 768px) {
  main section:nth-of-type(2) {
    grid-template-columns: repeat(2, 1fr);
  }
}

```

Ingat dari [dokumentasi jobsheet-02](#), selector `main section:nth-of-type(2)` menargetkan section "Ringkasan" (kartu statistik Total Buku/Anggota/Sedang Dipinjam) di Beranda, yang gaya dasarnya (di luar media query) adalah `grid-template-columns: repeat(3, 1fr)` — 3 kolom sama lebar. Di dalam breakpoint ini, nilai tersebut **ditimpa** menjadi `repeat(2, 1fr)` — 2 kolom. Karena selector-nya identik persis dengan gaya dasar (sama-sama `main section:nth-of-type(2)`, spesifikitas sama), aturan di dalam `@media` yang ditulis lebih akhir di file yang menang selama kondisi lebar layar terpenuhi (ingat urutan penulisan penting — sudah dibahas di [konsep dasar §1.5](#)).

Hasilnya: di layar tablet ($\leq 768\text{px}$), 3 kartu statistik yang tadinya sejajar dalam 1 baris kini tersusun 2 kolom (kartu ketiga otomatis turun ke baris kedua) — CSS Grid menangani pengaturan baris baru ini secara otomatis begitu jumlah kolom berkurang.

5.3 Breakpoint 2: Mobile ($\leq 480\text{px}$) — Perubahan Terbanyak

Blok kedua ini jauh lebih ramai karena di layar HP paling banyak penyesuaian yang diperlukan:

Aturan

```
header { position: relative; }
```

Efek

Dasar penempatan untuk `<nav>` yang terbuka — dibahas di [bab 3 §3.6](#).

Aturan	Efek
<code>.nav-toggle-label { display: block; }</code>	Ikon hamburger ☰ dimunculkan — dibahas di bab 3 §3.6 .
<code>header nav { display: none; ... }</code>	Menu disembunyikan secara default di layar mobile — dibahas di bab 3 §3.4 .
<code>.nav-toggle:checked ~ nav { display: block; }</code>	Menu dimunculkan saat hamburger diklik — inti checkbox hack, dibahas di bab 3 §3.5 .
<code>header nav ul { flex-direction: column; ... }</code>	Item menu tersusun vertikal saat terbuka — dibahas di bab 3 §3.7 .
<code>main section:nth-of-type(2) { grid-template-columns: 1fr; }</code>	Kartu statistik jadi 1 kolom (bertumpuk penuh vertikal) — lihat §5.4 .
<code>form input, form select { max-width: 100%; }</code>	Input form melebar penuh — lihat §5.5 .

Bagian menu hamburger sudah dibahas mendalam di [bab 3](#), jadi bab ini akan fokus ke dua baris terakhir yang belum dibahas.

5.4 Grid Kartu Statistik di Mobile: 1 Kolom

```
main section:nth-of-type(2) {
  grid-template-columns: 1fr;
}
```

Nilai `1fr` (hanya **satu** bagian pecahan) berarti grid ini sekarang hanya punya **1 kolom** yang lebarnya memenuhi seluruh ruang tersedia — efektif membuat ketiga kartu statistik (Total Buku, Total Anggota, Sedang Dipinjam) tersusun **bertumpuk vertikal**, satu di bawah satu, masing-masing selebar penuh layar. Ini progresi lengkap yang terjadi di 3 breakpoint berbeda:

Lebar Layar	grid-template-columns	Susunan Kartu
Desktop (>768px)	<code>repeat(3, 1fr)</code> (gaya dasar, jobsheet-02)	3 kolom sejajar
Tablet (≤768px)	<code>repeat(2, 1fr)</code> (§5.2)	2 kolom, kartu ke-3 turun baris

Lebar Layar	grid-template-columns	Susunan Kartu
Mobile ($\leq 480\text{px}$)	1fr	1 kolom, semua bertumpuk vertikal

Di layar sempit seperti HP, 3 kartu berdampingan akan membuat masing-masing kartu terlalu sempit untuk menampilkan angka besarnya dengan nyaman (ingat ukuran font `1.8rem` pada angka statistik dari [dokumentasi jobsheet-02 §6.6](#)) — menyusunnya vertikal memberi tiap kartu ruang penuh selebar layar.

5.5 Input Form Melebar Penuh di Mobile

```
form input,
form select {
  max-width: 100%;
}
```

Ingat dari [dokumentasi jobsheet-02 §8.4](#), gaya dasar input/select adalah `width: 100%; max-width: 400px;` — lebar penuh, tapi **dibatasi** maksimal 400px supaya tidak melebar berlebihan di layar desktop yang luas. Di breakpoint mobile ini, batas `max-width` tersebut **dilonggarkan** menjadi `100%` — di layar HP yang sudah sempit dengan sendirinya (jarang melebihi 400px), pembatasan 400px justru tidak diperlukan lagi dan malah bisa menyisakan ruang kosong janggal di kanan input kalau lebar layar sedikit di atas 400px. Perhatikan properti `width: 100%` dari gaya dasar **tidak perlu diulang** di sini — hanya `max-width` yang perlu ditimpa, karena `width: 100%` memang sudah berlaku sejak gaya dasar dan tidak bertentangan dengan tujuan breakpoint ini.

5.6 Cara Menguji Sendiri di Browser

1. Buka `index.html` (atau halaman lain) di browser.
2. Buka **DevTools** (klik kanan → *Inspect*, atau tombol `F12`).
3. Aktifkan **mode responsif/perangkat** (di Chrome: ikon HP/tablet di pojok kiri atas panel DevTools, atau `Ctrl+Shift+M`).
4. Ubah lebar layar secara bertahap dari lebar ($> 768\text{px}$) ke sempit ($< 480\text{px}$), amati:
 - Kartu statistik berubah dari 3 → 2 → 1 kolom tepat saat melewati 768px dan 480px.
 - Menu navigasi (di header) berubah jadi ikon `☰` begitu lebar $\leq 480\text{px}$.
 - Klik ikon `☰` tersebut — menu akan terbuka vertikal di bawah header (efek checkbox hack dari [bab 3](#)).
 - Buka `buku/list.html`, perkecil layar sampai tabelnya lebih lebar dari layar — coba scroll tabel itu ke samping.

6. Rangkuman & Latihan Lanjutan

6.1 Rangkuman Keseluruhan Jobsheet 3

Bagian	Lokasi Kode	Konsep yang Dipelajari
Konsep Dasar	—	Responsive design, <code><meta viewport></code> , media query, breakpoint, strategi desktop-first
Perubahan HTML	Semua file <code>.html</code>	Tag viewport, pasangan <code>input[checkbox] + label</code> , <code><div class="table-responsive"></code>
Hamburger Checkbox Hack	<code>.nav-toggle</code> , <code>.nav-toggle-label</code> , <code>.nav-toggle:checked</code> <code>~ nav</code>	Sibling combinator <code>~</code> , pseudo-class <code>:checked</code> , interaktivitas murni CSS tanpa JavaScript
Tabel Responsif	<code>.table-responsive</code>	<code>overflow-x: auto</code> , pola wrapper <code><div></code> untuk scroll horizontal
Media Query & Breakpoint	<code>@media (max-width: 768px)</code> , <code>@media (max-width: 480px)</code>	Sintaks <code>@media</code> , breakpoint tablet/mobile, override berbasis urutan penulisan

6.2 Konsep Inti yang Perlu Diingat

- `<meta name="viewport">` **adalah syarat wajib** untuk responsive design — tanpanya, seluruh media query di CSS tidak akan bekerja sebagaimana mestinya di HP karena browser mobile akan menganggap lebar layar 980px ([bab 1](#)).
- CSS bisa dibuat interaktif tanpa JavaScript**, memanfaatkan pseudo-class state bawaan seperti `:checked` (checkbox hack) yang dikombinasikan dengan sibling combinator `~` ([bab 3](#)). Ini teknik yang berguna untuk interaksi sederhana, meski untuk kasus lebih kompleks JavaScript tetap lebih fleksibel (disinggung di [README.md](#) jobsheet ini — akan diganti JavaScript di Jobsheet 5).
- `overflow-x: auto` **pada** `<div>` **pembungkus** adalah pola umum menangani tabel/konten lebar di layar sempit tanpa merusak tata letak halaman lain ([bab 4](#)).
- Media query menempa gaya dasar berdasarkan urutan penulisan** ketika spesifisitas selectornya sama — karena itu breakpoint di jobsheet ini sengaja diletakkan di **bagian**

paling bawah file ([bab 5](#)).

5. **Desktop-first vs mobile-first** adalah dua strategi menulis CSS responsif; jobsheet ini memakai desktop-first (`max-width`), lawan dari mobile-first (`min-width`) — keduanya valid, pilihannya tergantung kebiasaan tim/proyek ([bab 1 §1.5](#)).

6.3 Cara Mencoba Sendiri

1. Ikuti langkah pengujian DevTools di [bab 5 §5.6](#).
2. Di DevTools mode responsif, coba **ubah lebar layar perlahan** sambil mengamati kartu statistik — perhatikan tepat di angka 768px dan 480px susunannya "patah" ke jumlah kolom yang berbeda. Ini adalah efek breakpoint yang bekerja tepat di titik yang ditentukan.
3. Klik ikon hamburger  berkali-kali di mode mobile, amati bahwa menu terbuka/tertutup **tanpa reload halaman** — buktikan sendiri bahwa ini murni CSS dengan membuka tab **Console** DevTools; tidak akan ada log/error JavaScript apa pun yang terpicu dari interaksi ini.

6.4 Ide Latihan Tambahan (Opsional)

1. **Tambah breakpoint baru** — misalnya `@media (min-width: 1400px)` untuk layar monitor sangat lebar, ubah `main { max-width: 1000px; }` (dari [dokumentasi jobsheet-02](#)) menjadi lebih lebar khusus di breakpoint ini.
2. **Ubah breakpoint tablet** dari 768px menjadi 900px, lalu amati di lebar layar berapa susunan kartu berubah — buktikan bahwa breakpoint memang bisa disesuaikan bebas sesuai kebutuhan desain.
3. **Terapkan pola** `table-responsive` ke elemen lain yang berpotensi melebar di layar sempit, misalnya kalau suatu saat kamu menambahkan blok kode `<pre>` yang panjang di salah satu halaman.
4. **Ubah posisi ikon hamburger** — misalnya pindahkan `.nav-toggle-label` ke urutan terakhir di `<header>` (setelah `<nav>`) lalu amati apakah sibling combinator `.nav-toggle:checked ~ nav` di [bab 3 §3.5](#) masih bekerja — ingat catatan bahwa combinator `~` mensyaratkan target berada **setelah** elemen sumbernya di HTML.
5. **Bandingkan dengan pendekatan mobile-first** — coba tulis ulang `style.css` dari nol memakai `@media (min-width: ...)` alih-alih `max-width`, dan rasakan sendiri bedanya alur berpikirnya.

Kalau ada bagian yang masih membingungkan, terutama soal sibling combinator dan checkbox hack, coba baca ulang [bab 3](#) sambil mempraktikkan langsung di DevTools — konsep ini jauh lebih mudah dipahami dengan mencoba klik ikon hamburger sambil melihat panel **Elements** DevTools berubah menampilkan/menyembunyikan atribut `checked` pada checkbox-nya secara langsung.